1 Разбор заданий к лекции 16: Почему FPT-задача может не иметь полиномиального ядра

Условия. Авторские решения; номера совпадают с заданиями.

1.1 16.1

Возьмём два графа из одной изолированной вершины и порог $k = 2$. В каждом нет независимого множества размера 2, но в объединении оно есть. Independent Set может собирать решение по разным компонентам. Для k-Path это невозможно из-за связности пути. Для Vertex Cover минимум аддитивен: $\tau(G_1 \dot\cup G_2) = \tau(G_1) + \tau(G_2)$, поскольку каждый компонент нужно покрыть отдельно и объединение локальных оптимумов достаточно. При одном общем бюджете это не простой OR.

1.2 16.2

Размер результата ограничен $O((2^k)^3) = O(8^k)$, что экспоненциально по исходному k. Более того, заявленное полиномиальное время ядра по выходному входу не гарантирует полиномиальный размер по k. При двоичной записи 2^k занимает $k + 1$ бит, поэтому само значение параметра ещё не исключает полиномиальности преобразования по длине исходного унарного параметра. Но оно не удовлетворяет условию PPT $k' \le \mathrm{poly}(k)$.

При унарной записи k' уже требуется 2^k символов. Значит, такое преобразование не может быть полиномиальным на всех входах малого размера относительно k; оно может быть лишь FPT-преобразованием. В обеих моделях обещанного PPT-переноса нет.

1.3 16.3

1.3.1 Vertex Cover, k

Время: $O^*(1.4656^k)$.

Ядро / условный запрет: не более $2k$ вершин; нет $2^{o(k)}N^{O(1)}$ при ETH.

Основание: ветвление и LP-ядро доказаны; ETH-перенос использует внешнюю sparsification lemma.

1.3.2 k-Path, k вершин

Время: случайное $(2e)^k N^{O(1)}$; детерминированное $(2e)^k k^{O(\log k)} N^{O(1)}$.

Ядро / условный запрет: нет полиномиального ядра при NP $\not\subseteq$ coNP/poly.

Основание: вероятностная DP доказана; дерандомизация опирается на explicit hash families; запрет — на внешнюю теорему дистилляции.

1.3.3 Clique, k

Время: $n^k\,\mathrm{poly}(k)$.

Ядро / условный запрет: W[1]-полна; не FPT при FPT $\neq W[1]$.

Основание: полнота Clique — внешняя базовая теорема; редукции к другим задачам разобраны.

1.3.4 Independent Set, данное разложение ширины t

Время: $2^t N^{O(1)}$.

Ядро / условный запрет: существование некоторого ядра для разрешимой параметризованной decision-версии следует из FPT; полиномиальное ядро здесь не установлено.

Основание: DP доказана; если разложение не дано, его поиск требует внешнего алгоритма.

Последний случай не противоречит трудности Independent Set по размеру искомого множества. Упоминание некоторого ядра относится к decision-версии (есть ли независимое множество заданного размера), а оптимизационная DP решает её сравнением оптимума с порогом. В верхних оценках N включает кодировку разложения.

1.4 16.4

Корректность кодировки и тройку чисел можно вычислить за полиномиальное время; равенство троек — отношение эквивалентности. При длине строки до N и явной записи всех вершин имеем $n \le O(N)$, $b \le n$, $m \le n(n-1)/2 = O(N^2)$. Поэтому троек не больше $O(N^4)$, плюс один класс некорректных входов. Более плотная оценка здесь не требуется.

Группировка по точному списку рёбер разделяет все различные помеченные графы. Для n вершин существует $2^{\binom{n}{2}}$ графов, каждый имеет явную кодировку длины $O(n^2 \log(n+2))$ и допустимое фиксированное b. Число классов сверхполиномиально по этой длине. Проверяемость отношения сама по себе не гарантирует полиномиальное число классов; нужны оба условия.

1.5 16.5

Последовательно применяем PPT и ядро B. Ответ сохраняется, время полиномиально, размер результата $O(((k+1)^2+1)^3) = O((k+1)^6)$. Но результат — экземпляр B, поэтому пока доказана компрессия A в язык B, а не ядро той же задачи A.

При дополнительном обратном преобразовании применяем его к маленькому выходу ядра B. Если его время и полная длина результата ограничены полиномом степени c от длины этого выхода, получаем размер $O((k+1)^{6c})$ и полиномиальное суммарное время. Ответ попрежнему эквивалентен, выход теперь принадлежит формату A: это ядро. Условие о полной кодировке учитывает также параметр; нельзя полиномиально ограничить только основную строку, забыв огромный унарный параметр. Обратная редукция применяется после сжатия, поэтому даже её полиномиальное увеличение размера остаётся полиномиальным по k.